

why ten?

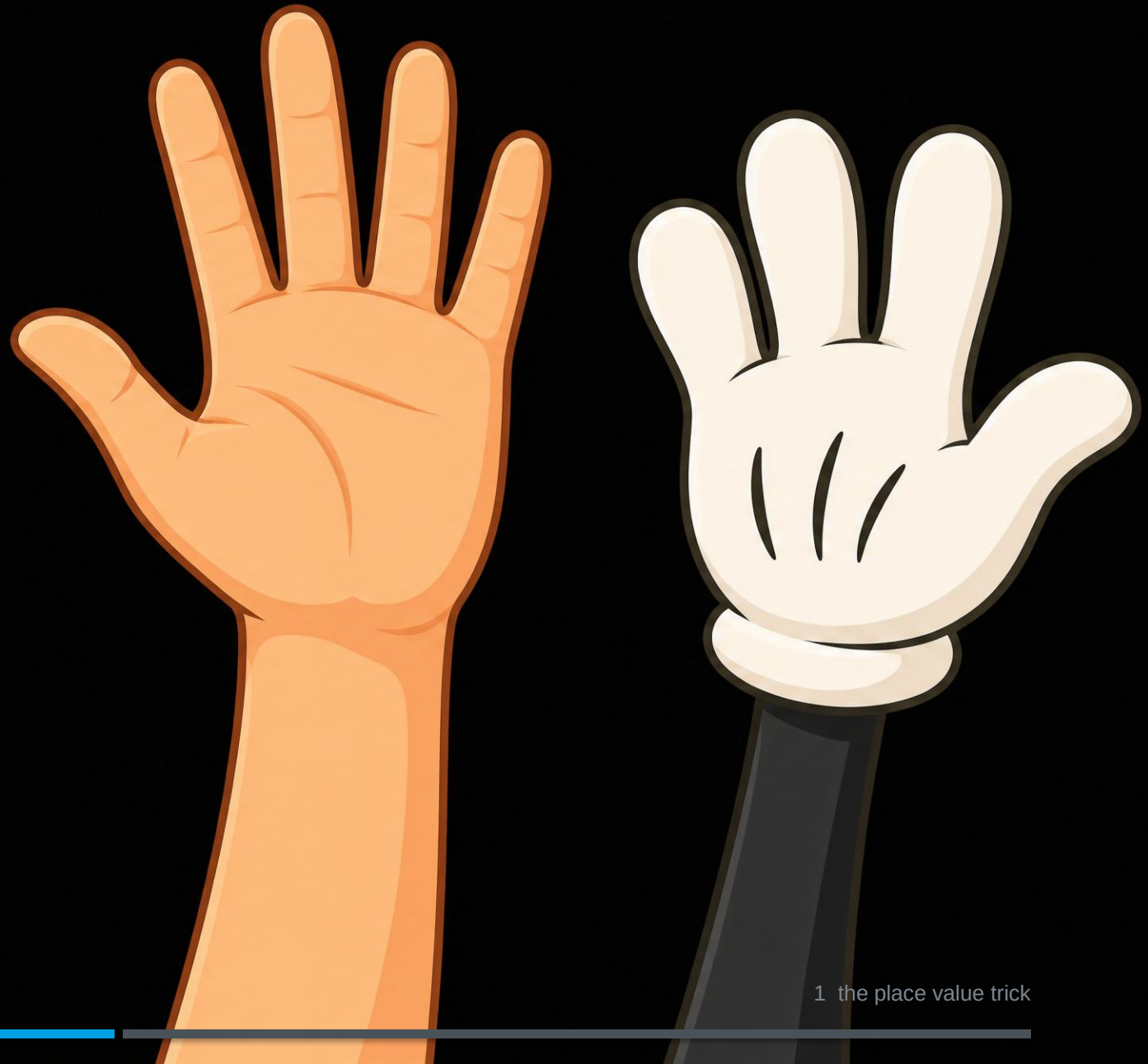
digitalisierung und programmierung · input, session 6

agenda

- 1 the place value trick
- 2 bits, bytes and hex

the place value trick

why do we
count to ten?



what 123 really says

1

2

3

10^2

10^1

10^0

100

10

1

what 123 really says

1	2	3
10^2	10^1	10^0
100	10	1

$$1 \times 100 + 2 \times 10 + 3 \times 1 = 123$$

the eight-finger creature writes 123

1

2

3

8^2

8^1

8^0

64

8

1

the eight-finger creature writes 123

1	2	3
8^2	8^1	8^0
64	8	1

$$1 \times 64 + 2 \times 8 + 3 \times 1 = 83$$

two flippers: counting in binary

we write

the dolphin writes

0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111



only two digits, so the positions overflow constantly: the same numbers 0 to 7.

the dolphin writes 110

1

1

0

2^2

2^1

2^0

4

2

1

$$1 \times 4 + 1 \times 2 + 0 \times 1 = 6$$

one question before the break

why do computers use base 2, of all things?

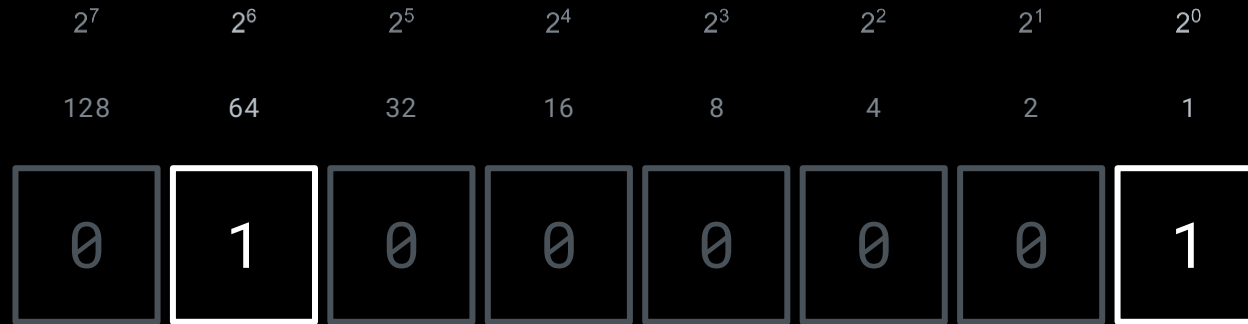
bits, bytes and hex

a word you already know

bit = binary digit

a binary digit holds two values: exactly one yes/no answer, exactly one bit of information. the container is named after its content.

eight bits make a byte



$$= 64 + 1 = 65$$

possible values

$$2^8 = 256$$

read as a number: 0 to 255.
your colour channels, exactly.

flip the bits yourself:

winf-hsos.github.io/lifi-concept-demos

eight bits make a byte. every position doubles the possibilities.

hex: base 16, two digits per byte

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
										10	11	12	13	14	15

F

16^1

16

F

16^0

1

one hex digit = 4 bits,
two digits = one byte.

#AC8909 is just
three bytes.

$$15 \times 16 + 15 \times 1 = 255$$

two ladders of units

decimal: each step x 1000

kilobyte	(KB)	10^3 bytes
megabyte	(MB)	10^6 bytes
gigabyte	(GB)	10^9 bytes
terabyte	(TB)	10^{12} bytes

binary: each step x 1024

kibibyte	(KiB)	2^{10} bytes
mebibyte	(MiB)	2^{20} bytes
gibibyte	(GiB)	2^{30} bytes
tebibyte	(TiB)	2^{40} bytes

why 1024? because memory is organised in powers of two, and $2^{10} = 1024$

two ladders, confusingly similar names: x 1000 and x 1024.

the missing 69 gigabytes

on the box: 1 TB

1 000 000 000 000 bytes

counted down the
decimal ladder, x 1000

your computer says: 931 GiB

the same bytes

counted down the
binary ladder, x 1024

nothing is missing: over four rungs the two ladders drift seven percent apart.

the point of all this

your colour alphabet is a place value system

k colours are k digits, and position matters. with four colours and three symbols: $4 \times 4 \times 4 = 64$ codes.

next up: what the bits mean.